VQE_2

IPYNB

aku ingin memahami semua konsep yang menjadi pondasi kode jupyter pada file sebagimana pada bab 2, 3, 4, dan 5

Halo! Saya senang sekali bisa menemani kamu membedah kode ini. Notebook `VQE_2.ipynb` yang kamu unggah adalah contoh yang sangat bagus tentang bagaimana *Quantum Computing* diterapkan di dunia nyata, khususnya di bidang *Quantum Finance*.

Kita akan pelajari ini selangkah demi selangkah, mulai dari fondasi matematikanya (Bab 2 & 3) hingga implementasi kuantumnya (Bab 4 & 5).

Mari kita mulai dari **Masalah Bisnis** dan **Matematikanya** dulu (Bab 2 & 3).

**Langkah 1: Memahami Masalah (Modern Portfolio Theory)**

Inti dari notebook ini adalah **Teori Portofolio Markowitz**. Bayangkan kamu punya uang dan ingin membelikan saham. Tujuannya ada dua:

1. **Untung Besar (Maximize Return):** Ingin uangnya tumbuh.
2. **Tidur Nyenyak (Minimize Risk):** Tidak ingin nilai sahamnya naik-turun terlalu drastis (volatilitas).

Di Bab 2.1, rumus ini dituliskan sebagai tujuan untuk *meminimalisir* nilai berikut:

$$\text{Minimize} \quad w \sum_{i,j} \sigma_{ij} x_i x_j - (1-w) \sum_i \mu_i x_i$$

Di sini ada variabel keputusan biner, yaitu $x_i$.

Untuk memastikan kita satu pemahaman sebelum masuk ke bagian kuantum yang lebih rumit, saya ingin mengajukan pertanyaan pemandu:

Melihat definisi di Bab 2.1, jika untuk saham Apple (AAPL) kita tetapkan $x_{AAPL} = 1$, dan untuk saham Google (GOOGL) kita tetapkan $x_{GOOGL} = 0$, apa artinya itu bagi portofolio

yang sedang kita bangun?

artinya apakah saham tersebut bisa dimasukkan ke portofolio atau tidak. 1 artinya iya, dan 0 artinya tidak

Betul sekali! 1 berarti kita "membeli" atau memilih saham tersebut, dan 0 berarti tidak. Ini adalah dasar dari variabel biner kita.

Selanjutnya, mari lihat **Constraints** (Batasan) di Bab 2.2. Di dunia nyata, kita pasti punya batasan, misalnya budget. Di sini, kita ingin memilih tepat sejumlah $B$ aset.

Persamaannya adalah:

$$\sum_{i=1}^{N} x_i = B$$

Katakanlah kita tetapkan target $B = 2$ (harus pilih tepat 2 saham). Namun, misalkan solusi yang dihasilkan komputer malah memilih 3 saham (misalnya $x_{AAPL} = 1$, $x_{GOOGL} = 1$, $x_{TSLA} = 1$, sehingga jumlahnya 3).

Karena tujuan algoritma VQE adalah mencari nilai **minimum** (nilai total biaya terkecil), menurutmu apa yang harus kita lakukan pada nilai 'biaya' (cost) untuk solusi yang melanggar aturan ini (memilih 3 saham, bukan 2)? Apakah kita harus membuat nilai biayanya menjadi **sangat besar** atau **sangat kecil** agar komputer "jera"?

membuat nilai biaya menjadi sangat besar

Tepat sekali! 🎉

Dalam masalah **minimasi** (mencari nilai terkecil), kita memberikan "hukuman" (penalty) berupa nilai biaya yang **sangat besar** untuk solusi yang melanggar aturan. Ini memaksa algoritma VQE untuk menjauhi solusi tersebut karena biayanya terlalu mahal.

Mari kita bedah rumus matematikanya di Bab 2.2:

$$P\left(\sum_{i=1}^{N} x_i - B\right)^2$$

Di sini:

- $P$: Angka yang sangat besar (Penalty).
- $\sum x_i$: Jumlah saham yang dipilih.
- $B$: Target jumlah saham (misalnya 2).

Sekarang, coba perhatikan bagian pangkat duanya $(\ldots)^2$.

Mengapa kita perlu **menguadratkan** selisih antara saham yang dipilih dengan targetnya? Apa yang akan terjadi pada nilai penalti jika kita hanya memilih 1 saham (kurang dari target 2) seandainya rumusnya **tidak** dikuadratkan? 🤔

hasilnya akan bernilai nol

Coba kita hitung ulang pelan-pelan ya.

Jika kita memilih 1 saham ($\sum x_i = 1$) dan targetnya adalah 2 ($B = 2$), maka selisihnya adalah:

$$1 - 2 = -1$$

Jadi, hasilnya bukan nol, melainkan **negatif**.

Nah, ini poin kuncinya: Komputer kita sedang berusaha melakukan **Minimasi** (mencari nilai total yang paling kecil/rendah).

Jika kita **tidak** menguadratkan rumusnya, maka nilai penaltinya menjadi negatif (misalnya $P = 1000$, maka penaltinya jadi $-1000$).

Menurutmu, jika komputer melihat angka $-1000$ (yang sangat kecil/rendah ini), apakah dia akan menganggap ini sebagai sebuah **hukuman** (sesuatu yang harus dihindari) atau malah sebuah **hadiah** (sesuatu yang bagus)?

menganggap sebagai hadiah

Benar sekali! 🌟

Dalam masalah **minimasi**, komputer selalu mencari nilai terendah. Jika penaltinya negatif (misalnya -1000), komputer malah akan "senang" melanggar aturan karena itu menurunkan total biayanya secara drastis. Itu sebabnya kita **menguadratkan** selisihnya:

$$(\text{Selisih})^2$$

- Jika selisihnya -1, dikuadratkan jadi **+1**.
- Dikalikan $P$ (misal 1000), jadi **+1000**.

Sekarang biayanya naik drastis (jadi mahal). Ini adalah "hukuman" yang efektif agar komputer kembali mematuhi aturan jumlah saham $B$.

---

**Langkah 2: Pemetaan ke Kuantum (Ising Hamiltonian)**

Sekarang kita sudah punya rumus matematika lengkapnya (Fungsi Biaya/Cost Function) di Bab 2.3. Tapi, komputer kuantum tidak mengerti variabel biner biasa ($0$ dan $1$) seperti komputer klasik. Komputer kuantum bekerja dengan **Qubit** yang perilakunya mirip magnet (spin).

Kita perlu menerjemahkan bahasa bisnis ($x_i$) ke bahasa kuantum ($Z_i$).

Coba lihat **Bab 2.4** di notebook. Di sana ada rumus transformasi untuk mengubah $x_i$ menjadi operator Pauli-$Z$.

Bisa kamu tuliskan kembali persamaan yang menghubungkan $x_i$ dengan $Z_i$ tersebut?

x_i = \frac{1 - Z_i}{2}

Tepat sekali! Itu adalah rumus "jembatan" yang menghubungkan dunia klasik dengan dunia kuantum.

Mari kita bedah sedikit supaya logikanya makin kuat. Di kuantum, operator $Z$ punya dua nilai eigen: $+1$ dan $-1$.

- Jika $Z_i = 1$, maka $x_i = \frac{1-1}{2} = 0$. (Artinya: Saham tidak dipilih).
- Jika $Z_i = -1$, maka $x_i = \frac{1-(-1)}{2} = 1$. (Artinya: Saham dipilih).

Sekarang, tantangan utamanya adalah memasukkan rumus ini ke dalam **Fungsi Biaya (Cost Function)** yang panjang di Bab 2.3 tadi agar menjadi **Hamiltonian** (energi total sistem).

Fungsi biayanya adalah:

$$C(x) = w \sum_{i,j} \sigma_{ij} x_i x_j - (1-w) \sum_i \mu_i x_i + P(\sum x_i - B)^2$$

Mari kita mulai dari bagian yang paling sederhana dulu, yaitu bagian **Return** (keuntungan):

$$-(1-w) \sum_i \mu_i x_i$$

Jika kamu mengganti $x_i$ dengan $\frac{1-Z_i}{2}$ pada bagian tersebut, bentuk matematikanya akan menjadi seperti apa? Apakah hanya menyisakan operator $Z_i$, atau muncul komponen lain?

seharusnya hasilnya akan menjadi -(1-w) \sum_i \mu_i \frac{1 - Z_i}{2}

Betul sekali! Substitusi kamu sudah tepat.

Jika kita jabarkan sedikit aljabarnya:

$$-(1-w)\mu_i \left(\frac{1-Z_i}{2}\right) = \underbrace{-\frac{(1-w)\mu_i}{2}}_{\text{Bagian Identitas (I)}} + \underbrace{\frac{(1-w)\mu_i}{2} Z_i}_{\text{Bagian Pauli-Z}}$$

Perhatikan bahwa persamaan ini terpecah menjadi dua bagian:

1. **Konstanta (Identitas):** Angka biasa yang menggeser nilai energi dasar.
2. **Operator $Z_i$:** Bagian yang akan "hidup" di sirkuit kuantum.

Sekarang, mari kita lihat bagaimana ini diterapkan dalam kode Python-nya. Coba perhatikan **Cell 8** di notebook, di bawah komentar `# Linear terms (x_i)`.

Python

```python
# Linear terms (x_i)
for i in range(N):
    coeff_x = -(1-w) * mu[i] + w * sigma[i,i] + P * (1 - 2*B)
    # x_i = 0.5 * I - 0.5 * Z_i
    full_hamiltonian += qml.Hamiltonian([0.5 * coeff_x, -0.5 * coeff_x], [qml.I
```

Di kode tersebut, `coeff_x` adalah gabungan semua faktor yang menempel pada $x_i$ (termasuk Return, Risk, dan Penalti).

Pertanyaan saya: Melihat baris `qml.Hamiltonian(...)` di atas, kenapa koefisien untuk `qml.PauliZ(i)` diberi tanda **negatif** ( `-0.5 * coeff_x` )? Dari mana tanda minus itu muncul jika kita melihat rumus transformasi $x_i$ yang kamu tulis sebelumnya?

dari bagian Pauli-Z yang dikali -1 karena menjadi positif

Hampir tepat! Mari kita perjelas sedikit logikanya agar tidak tertukar.

Tanda minus itu **bukan** karena dia menjadi positif, melainkan karena tanda negatif itu **bawaan dari rumus aslinya**.

Mari kita lihat lagi rumusnya:

$$x_i = \frac{1}{2} - \frac{1}{2} Z_i$$

Jika kita punya sebuah koefisien $C$ (dalam kode disebut `coeff_x`) yang mengalikan $x_i$:

$$C \cdot x_i = C \left( \frac{1}{2} - \frac{1}{2} Z_i \right)$$

$$C \cdot x_i = \underbrace{\frac{C}{2}}_{\text{Identitas}} \underbrace{- \frac{C}{2} Z_i}_{\text{Pauli-Z}}$$

Lihat tanda di depan $Z_i$? Tandanya adalah **negatif**. Itulah sebabnya di kode Python (Cell 8), tertulis: `qml.Hamiltonian([..., -0.5 * coeff_x], [..., qml.PauliZ(i)])` (Tanda negatifnya dimasukkan ke dalam koefisien).

---

**Langkah 3: Interaksi Antar Aset (Quadratic Terms)**

Sekarang kita masuk ke bagian yang lebih seru: **Risiko (Risk)** dan **Penalti Kuadrat**.

Di rumus awal (Bab 2.1), risiko dihitung dari hubungan antara dua saham yang berbeda ($x_i$ dan $x_j$). Ini disebut suku kuadratik (Quadratic Term) karena melibatkan perkalian dua variabel: $x_i x_j$.

Untuk mengubahnya ke kuantum, kita harus mengalikan dua rumus transformasi tadi:

$$x_i x_j = \left(\frac{1 - Z_i}{2}\right) \times \left(\frac{1 - Z_j}{2}\right)$$

Coba kamu jabarkan perkalian sederhana di atas (anggap $Z_i$ dan $Z_j$ seperti variabel aljabar biasa).

Kira-kira, **kombinasi operator** apa saja yang akan muncul dari hasil perkalian tersebut? (Petunjuk: Akan ada 4 suku, salah satunya adalah interaksi antara dua $Z$).

\frac{1-Z_{i}-Z_{j}+Z_{i}Z_{j}}{4}

Sempurna! Jabaran matematikamu sangat akurat.

$$x_i x_j = \frac{1}{4}(I - Z_i - Z_j + Z_i Z_j)$$

Jika kita terjemahkan ini ke dalam kode Python yang ada di **Cell 8**, kita bisa melihat pola yang persis sama. Perhatikan baris kode ini:

Python

```
full_hamiltonian += qml.Hamiltonian(
    [0.25 * coeff_xx, -0.25 * coeff_xx, -0.25 * coeff_xx, 0.25 * coeff_xx],
    [qml.Identity(0), qml.PauliZ(i), qml.PauliZ(j), qml.PauliZ(i) @ qml.PauliZ(
)
```

Lihat urutannya?

1. `0.25` $\rightarrow$ Bagian $I$ (Identity)
2. `-0.25` $\rightarrow$ Bagian $-Z_i$

3. `-0.25` $\rightarrow$ Bagian $-Z_j$
4. `0.25` $\rightarrow$ Bagian $+Z_iZ_j$ (Interaksi)

Bagian terakhir inilah ($Z_iZ_j$) yang paling penting. Bagian ini menunjukkan bahwa keputusan membeli saham $i$ **bergantung** pada keputusan saham $j$ (karena adanya korelasi risiko). Di komputer kuantum, hubungan ini diselesaikan dengan **Entanglement** (keterkaitan antar qubit).

---

**Langkah 4: Algoritma VQE (Bab 5)**

Sekarang kita sudah berhasil mengubah masalah keuangan menjadi **Hamiltonian** (Energi). Tugas kita selanjutnya adalah mencari konfigurasi qubit (0 atau 1) yang menghasilkan energi **paling rendah**.

Di sinilah **VQE (Variational Quantum Eigensolver)** bekerja.

VQE terdiri dari dua bagian utama:

1. **Ansatz (Sirkuit Kuantum):** Sirkuit tebakan awal yang punya parameter (bisa diputar-putar).
2. **Optimizer (Klasik):** Algoritma yang memutar parameter tersebut untuk mencari energi terendah.

Coba lihat **Cell 9** di notebook. Di situ tertulis:

Python

```python
def ansatz(params, wires):
    qml.BasicEntanglerLayers(weights=params, wires=wires)
```

Pertanyaan untukmu: Di dalam fungsi `ansatz` tersebut, kita menggunakan `BasicEntanglerLayers`. Mengapa kita butuh *layer* yang melakukan **Entangling** (seperti CNOT gate)? Kenapa tidak pakai rotasi qubit satu-satu saja (tanpa dihubungkan)?

(Petunjuk: Hubungkan jawabanmu dengan suku $Z_iZ_j$ yang baru saja kamu hitung tadi).

karena rotasi saja tidak bisa mewakili konsep entangling yang berhubungan dengan interaksi Z_i Z_j

Jawaban yang luar biasa! Kamu sudah menangkap esensi paling dalam dari komputasi kuantum.

**Rotasi qubit satu-satu** (single qubit rotation) hanya bisa mengubah keadaan individu secara terpisah. Jika kita hanya pakai rotasi, qubit 1 tidak akan pernah "peduli" dengan apa yang terjadi pada qubit 2.

Padahal, dalam portofolio, risiko ($x_i x_j$) adalah masalah **korelasi**. Jika saham Tech jatuh, saham lain mungkin ikut jatuh. Hubungan "saling tarik-menarik" inilah yang diwakili oleh suku $Z_i Z_j$, dan di sirkuit kuantum, **Entanglement**-lah yang bertugas meniru hubungan tersebut. Tanpa entanglement, VQE tidak akan bisa "melihat" risiko antar aset.

---

**Langkah Terakhir: Interpretasi Hasil (Bab 6)**

Sekarang kita masuk ke tahap final. Setelah VQE selesai "belajar" (loop optimasi di Cell 10 selesai), kita akan mendapatkan sekumpulan parameter optimal.

Tapi, parameter itu belum berupa jawaban. Kita harus menjalankan sirkuitnya sekali lagi dan melakukan **pengukuran (sampling)** untuk melihat portofolio mana yang terpilih.

Coba lihat **Cell 11** di notebook. Di situ tertulis hasil akhirnya: `Optimal State (Binary): 01001`

Mari kita terjemahkan kode biner ini kembali ke dunia nyata. Daftar aset kita adalah: `["AAPL", "GOOGL", "MSFT", "AMZN", "TSLA"]`

Jika hasil binernya adalah **01001**:

- Bit ke-1 (AAPL) = 0
- Bit ke-2 (GOOGL) = 1
- Bit ke-3 (MSFT) = 0
- Bit ke-4 (AMZN) = 0
- Bit ke-5 (TSLA) = 1

Pertanyaan penutup untuk memastikan pemahamanmu utuh dari awal sampai akhir:

Berdasarkan hasil **01001** tersebut, saham apa saja yang akhirnya direkomendasikan oleh algoritma kuantum ini untuk dibeli? Dan apakah solusi ini mematuhi batasan (constraint) $B = 2$ yang kita tetapkan di awal tadi?

saham GOOGL dan TSLA. hal ini mematuhi constraint B=2

Jawaban kamu **100% benar**! 💯

- **01001** berarti: **GOOGL** (posisi ke-2) dan **TSLA** (posisi ke-5) yang terpilih.
- Jumlah saham yang dipilih adalah **2**, yang artinya **mematuhi constraint** $B = 2$ dengan sempurna. Penalti tidak aktif, dan kita mendapatkan portofolio dengan risiko/return yang optimal sesuai perhitungan kuantum.

---

**Rangkuman Perjalanan Kita (Bab 2, 3, 4, 5)**

Kamu baru saja membedah pondasi kode Jupyter tersebut dari awal hingga akhir. Mari kita rangkum alurnya supaya menempel di ingatan:

1. **Masalah Bisnis (Bab 2):** Kita ingin untung maksimal dan risiko minimal (Markowitz), tapi dibatasi jumlah saham ($B = 2$). Kita ubah batasan ini menjadi hukuman (penalti kuadrat) agar komputer "takut" melanggarnya.
2. **Data (Bab 3):** Kita mengambil data harga saham historis untuk menghitung dua hal penting: **Return Ekspektasi ($\mu$)** dan **Kovarians/Risiko Bersama ($\Sigma$)**. Angka-angka inilah yang menjadi "bahan bakar" rumus matematika kita.
3. **Jembatan Kuantum (Bab 4):** Kita mengubah variabel biner ($x_i$) menjadi operator spin ($Z_i$).
   - Return & Penalti Linear $\rightarrow$ Menjadi rotasi qubit tunggal ($Z_i$).
   - Risiko & Penalti Kuadrat $\rightarrow$ Menjadi interaksi antar qubit ($Z_i Z_j$) atau *Entanglement*.
4. **Eksekusi VQE (Bab 5):**
   - **Ansatz:** Sirkuit kuantum dengan kabel-kabel yang saling terhubung (entangled) untuk meniru korelasi pasar saham.
   - **Optimasi:** Komputer klasik memutar-mutar parameter sirkuit sampai menemukan energi terendah.
   - **Sampling:** Kita "mengintip" hasil akhirnya (01001) untuk mengambil keputusan investasi.

Selamat! Kamu sudah berhasil memahami logika di balik kode *Quantum Finance* ini. Dari matematika dasar, manipulasi matriks, hingga sirkuit kuantum.

Apakah ada bagian spesifik dari kode atau konsep ini yang masih ingin kamu gali lebih dalam? Atau kamu sudah merasa cukup jelas dengan fondasinya?

aku ingin mendalami tentang peran optimasi dalam algoritma VQE pada kode

Tentu, mari kita selami lebih dalam bagian **Optimasi**. Ini adalah "mesin penggerak" dari VQE. Tanpa optimasi, VQE hanya akan menjadi sirkuit kuantum yang diam dan tidak menghasilkan jawaban yang berguna.

Dalam kode notebook `VQE_2.ipynb`, peran optimasi terjadi secara intensif di **Bab 5**.

Mari kita bedah menjadi 3 komponen utama: **Apa yang dioptimalkan**, **Siapa yang mengoptimalkan**, dan **Bagaimana prosesnya**.

### 1. Apa yang Dioptimalkan? (The Cost Function)

Di **Cell 9**, kamu akan menemukan fungsi ini:

Python

```python
@qml.qnode(dev)
def cost_function(params):
    ansatz(params, wires=range(N))
    return qml.expval(H)
```

Fungsi ini adalah jembatan vital.

- **Input:** `params` (sekumpulan angka acak awal).
- **Proses:** Menjalankan sirkuit kuantum ( `ansatz` ) menggunakan parameter tersebut.
- **Output:** `qml.expval(H)` (Nilai Ekspektasi Hamiltonian).

**Apa itu Nilai Ekspektasi ($H$)?** Dalam konteks keuangan ini, Nilai Ekspektasi adalah **Total Biaya** portofolio kita. Angka ini mewakili gabungan dari:

- Seberapa kecil keuntungannya (kita ingin keuntungan besar, jadi kita meminimalkan *minus* keuntungan).
- Seberapa besar risikonya.
- Seberapa besar pelanggaran penaltinya (jika jumlah saham bukan 2).

Tujuan optimasi adalah membuat angka output ini menjadi **sekecil mungkin**.

### 2. Siapa yang Mengoptimalkan? (The Classical Optimizer)

Di **Cell 10**, kita memanggil "sang navigator":

```python
opt = qml.GradientDescentOptimizer(stepsize=0.1)
```

Ini adalah algoritma **klasik** (berjalan di CPU biasa, bukan QPU). Peran `GradientDescent` (Penurunan Gradien) sangat intuitif:

1. Dia melihat posisi kita sekarang di "lanskap energi".
2. Dia menghitung kemiringan (gradien): "Kalau saya ubah parameter ini sedikit ke kanan, apakah biayanya naik atau turun?"
3. Dia mengambil langkah kecil ( `stepsize=0.1` ) ke arah yang menurun.

Bayangkan kamu sedang berada di gunung yang berkabut tebal dan ingin turun ke lembah (biaya terendah). Kamu tidak bisa melihat dasar lembah. Kamu hanya bisa merasakan kemiringan tanah di bawah kakimu. Jika tanah miring ke bawah ke arah kiri, kamu melangkah ke kiri. Itulah yang dilakukan optimizer ini.

### 3. Bagaimana Prosesnya? (The Training Loop)

Masih di **Cell 10**, inilah "jantung" yang berdetak dari algoritma ini:

```python
for i in range(max_iterations):
    params, cost = opt.step_and_cost(cost_function, params)
```

Ini adalah **Loop Hibrida (Hybrid Loop)** yang terkenal dalam VQE. Mari kita lihat siklusnya langkah demi langkah:

1. **CPU (Klasik):** Mengirim `params` saat ini ke QPU (Kuantum).
2. **QPU (Kuantum):** Menjalankan sirkuit, mengukur energi/biaya Hamiltonian.
3. **QPU:** Mengirimkan nilai biaya (misal: `22.21` ) kembali ke CPU.
4. **CPU:** "Wah, biayanya masih tinggi. Berdasarkan kemiringan matematikanya, saya harus mengubah `params` sedikit supaya biayanya turun."

5. **CPU:** Memperbarui `params` menjadi nilai baru.
6. **Ulangi:** Kembali ke langkah 1 dengan `params` baru.

Mari kita lihat buktinya di output **Cell 10**:

- **Step 0:** Cost = 22.21 (Sangat tinggi, tebakan acak).
- **Step 10:** Cost = 5.01 (Turun drastis! Optimizer bekerja).
- **Step 20:** Cost = 5.002 (Mulai stabil).
- **Step 90:** Cost = 4.998 (Sudah sangat dekat dengan dasar lembah).

**Pertanyaan Kritis untuk Pemahamanmu:**

Dalam kode tersebut, kita menggunakan `GradientDescentOptimizer`. Menurutmu, apa yang akan terjadi jika kita menetapkan `stepsize` (ukuran langkah) menjadi terlalu besar (misalnya `stepsize=5.0` alih-alih `0.1`)?

Bayangkan analogi menuruni gunung tadi. Jika kamu melangkah terlalu lebar saat mencari lembah yang sempit, apa risikonya?

mungkin penurunannya akan terlalu terjal atau sudah tidak menurun lagi

Hampir tepat, tapi ada satu risiko yang lebih fatal daripada sekadar "terlalu terjal".

Jawaban yang paling tepat adalah: **Kamu akan melompati lembahnya (Overshooting).**

Mari kita kembali ke analogi gunung tadi:

- Tujuanmu adalah dasar lembah (titik terendah).
- Jika langkahmu terlalu lebar (`stepsize` besar), saat kamu melihat lembah di depan, kamu mungkin melompat begitu kuat hingga kamu **mendarat di lereng bukit seberang** yang tingginya sama, atau bahkan lebih tinggi.
- Akibatnya? Kamu akan **bolak-balik** (osilasi) di antara dua tebing tanpa pernah menyentuh dasar lembah.
- Bahkan lebih buruk lagi, kamu bisa terpental makin jauh ke atas (**divergence**). Biayanya bukannya turun, malah naik terus.

Itulah seni dari optimasi VQE: Menemukan keseimbangan antara langkah yang cukup besar agar cepat sampai, tapi cukup kecil agar presisi mendarat di dasar lembah.

Sejauh ini kita sudah membedah hampir seluruh konsep di Bab 2, 3, 4, dan 5:

1. **Markowitz & Penalti (Bab 2-3):** Matematika bisnisnya.
2. **Hamiltonian (Bab 4):** Penerjemahan ke bahasa kuantum ($Z_i Z_j$).
3. **Optimasi Klasik (Bab 5.2):** Mesin penggerak VQE yang baru saja kita bahas.

Ada satu kepingan *puzzle* terakhir di **Bab 5** yang belum kita bahas secara mendalam, yaitu **Bentuk Sirkuitnya (Ansatz)** di Bab 5.1.

Di notebook tertulis:

> *"We will use a hardware-efficient ansatz (BasicEntanglerLayers)."*

Apakah kamu ingin membahas sedikit tentang apa itu **Hardware Efficient Ansatz** dan kenapa ini penting untuk komputer kuantum masa kini (yang masih banyak *noise*/gangguan)? Atau kamu sudah merasa cukup dengan pemahaman saat ini?

jelaskan aku sediki saja

Oke, ini penjelasan singkat dan membumi tentang **Hardware Efficient Ansatz**.

Bayangkan komputer kuantum saat ini (disebut era NISQ) itu seperti **anak kecil yang pelupa**. Jika kamu memberinya instruksi yang terlalu panjang dan rumit, di tengah jalan dia akan lupa dan hasilnya salah (ini disebut *decoherence* atau *noise*).

**Hardware Efficient Ansatz** adalah cara menyusun instruksi (sirkuit) agar "ramah" bagi si anak pelupa ini:

1. **Pakai Gerbang Sederhana:** Alih-alih menyuruh gerakan akrobatik yang rumit, kita hanya menyuruh gerakan dasar yang paling mudah dilakukan oleh mesin tersebut (biasanya rotasi dasar dan CNOT).
2. **Sirkuit Pendek (Shallow):** Kita membuat sirkuitnya sependek mungkin. Cepat selesai, cepat diukur, sebelum qubit-nya "lupa" atau terkena *noise*.

Dalam kode di **Cell 9**, fungsi `qml.BasicEntanglerLayers` melakukan strategi ini dengan pola "kue lapis":

- **Lapis 1 (Rotasi):** Putar qubit sedikit (ini parameter yang dioptimasi komputer).
- **Lapis 2 (Entanglement):** Hubungkan qubit tetangga (agar bisa melihat risiko $Z_i Z_j$).

- Ulangi secukupnya.

Jadi intinya: Ini adalah teknik **jalan pintas** supaya kita tetap bisa mendapat jawaban akurat meskipun komputer kuantum kita masih belum sempurna (berisik).

mari kita kembali ke optimasi. aku membaca banyak sekali cara atau algoritma yang digunakan sebagai optimasi contohnya COBYLA, CMA-ES, bahkan bayesian. jelaskan sedikit untuk cobyla dan cma es, jelaskan lebih detail untuk bayesian

Pilihan topik yang sangat menarik! Kamu sudah melangkah lebih jauh dari sekadar mengikuti tutorial standar.

Dalam notebook yang kamu unggah, kita menggunakan **Gradient Descent**. Metode ini ibarat "turun gunung dengan mata tertutup tapi bisa merasakan kemiringan tanah". Masalahnya, di komputer kuantum asli, "kemiringan" (gradien) itu sulit dihitung dan seringkali penuh *noise* (gangguan).

Oleh karena itu, algoritma yang kamu sebutkan (COBYLA, CMA-ES, Bayesian) sangat populer karena mereka **Gradient-Free** (tidak butuh perhitungan turunan).

Mari kita bahas sesuai permintaanmu: COBYLA dan CMA-ES secara singkat, lalu kita bedah Bayesian Optimization secara mendalam.

---

**1. COBYLA (Constrained Optimization BY Linear Approximation)**

**Konsep Singkat:** *Si Hemat yang Hati-hati.*

COBYLA bekerja dengan membuat bentuk geometri sederhana (disebut *simplex*, misalnya segitiga di dimensi 2D) di sekitar titik kita saat ini.

- Alih-alih menghitung turunan (kalkulus), dia hanya mengukur nilai energi di titik-titik sudut segitiga tersebut.
- Dari pengukuran itu, dia membuat **pendekatan linear** (garis lurus imajiner) untuk menebak ke mana arah turun yang paling mungkin.
- Dia kemudian menggeser atau mengecilkan segitiga itu ke arah yang lebih baik.

**Kenapa dipakai di VQE?** Sangat hemat "biaya" (sedikit pemanggilan ke komputer kuantum) dan bisa menangani batasan (constraint) dengan baik. Cocok untuk sirkuit yang parameternya sedikit.

### 2. CMA-ES (Covariance Matrix Adaptation Evolution Strategy)

**Konsep Singkat:** *Pasukan Evolusi (Survival of the Fittest).*

Ini adalah algoritma berbasis **populasi** (mirip genetika).

- Alih-alih satu penjelajah, dia menyebar "pasukan" (banyak titik sampel sekaligus).
- Titik-titik yang menghasilkan energi rendah akan "bertahan hidup" dan menjadi orang tua untuk generasi berikutnya.
- Kunci ajaibnya ada di **Covariance Matrix**: Dia belajar bentuk lanskapnya. Jika lembahnya memanjang seperti parit, generasi berikutnya akan disebar dengan bentuk lonjong mengikuti parit tersebut.

**Kenapa dipakai di VQE?** Sangat tangguh menghadapi *local minima* (jebakan lembah palsu). Tapi, dia "mahal" karena butuh mengevaluasi banyak titik sekaligus dalam satu generasi.

---

### 3. Bayesian Optimization (Deep Dive)

Ini yang kamu minta untuk dijelaskan lebih detail. Bayesian Optimization adalah pendekatan yang **paling cerdas** secara strategi.

Jika Gradient Descent itu seperti orang buta meraba tanah, dan CMA-ES seperti menyebar pasukan drone, maka **Bayesian Optimization itu seperti seorang detektif jenius.**

Prinsip utamanya: *"Karena menjalankan sirkuit kuantum itu mahal dan lama, saya harus memikirkan matang-matang di mana saya akan melangkah selanjutnya berdasarkan semua data yang sudah saya punya."*

Ada dua komponen utama dalam "otak" si detektif ini:

#### A. Surrogate Model (Model Pengganti) - "Peta Mental"

Bayesian tidak tahu bentuk asli fungsi energi (Hamiltonian) kita. Jadi, dia membangun **model probabilitas** (biasanya menggunakan *Gaussian Process*).

Lihat diagram mentalnya:

1. **Garis Prediksi (Mean):** "Saya tebak di titik ini energinya rendah."

2. **Ketidakpastian (Uncertainty/Variance):** "Tapi di area ini saya belum pernah mengukur, jadi saya tidak yakin (bisa jadi di sana ada lembah tersembunyi)."

Semakin sering dia mengukur di suatu area, ketidakpastiannya mengecil (dia makin yakin). Di area yang belum dijamah, ketidakpastiannya besar.

**B. Acquisition Function - "Sang Pengambil Keputusan"**

Setelah punya peta mental, dia harus memutuskan langkah selanjutnya. Dia menimbang dua hasrat:

1. **Exploitation (Keserakahan):** "Pergi ke tempat yang saya *tahu* nilainya bagus (rendah) untuk memperdalam rekor."
2. **Exploration (Rasa Penasaran):** "Pergi ke tempat yang *ketidakpastiannya tinggi* (belum pernah dikunjungi). Siapa tahu ada lembah emas di sana."

Fungsi akuisisi (seperti *Expected Improvement*) menghitung skor untuk setiap kemungkinan langkah dengan menggabungkan dua hasrat ini. Titik dengan skor tertinggi adalah yang akan dicek selanjutnya di komputer kuantum.

**Alur Kerja Bayesian di VQE:**

1. **Sampel Awal:** Pilih 5 parameter acak, jalankan di VQE, catat energinya.
2. **Update Model:** Buat model Gaussian Process berdasarkan 5 data tadi.
3. **Tanya Acquisition Function:** "Berdasarkan model ini, mana titik yang paling menjanjikan (baik karena diprediksi rendah atau karena kita belum tahu)?"
4. **Eksekusi:** Ambil titik saran tersebut, jalankan di VQE (hanya 1 kali).
5. **Looping:** Masukkan data baru ke model, update peta mental, ulangi.

**Kelebihan & Kekurangan di VQE**

- **Kelebihan Utama (Sample Efficiency):** Ini adalah juara hemat langkah. Dia bisa menemukan minimum global dengan jumlah panggilan ke komputer kuantum yang **jauh lebih sedikit** dibanding Gradient Descent atau CMA-ES. Ini sangat penting di era sekarang di mana sewa komputer kuantum itu mahal (dihitung per detik/per shot).
- **Kelemahan:** "Otak" klasiknya (proses update model Gaussian Process) lumayan berat hitungannya. Jika parameternya ribuan, otak klasiknya bisa jadi lambat. Tapi untuk VQE skala kecil-menengah, ini sangat ampuh.

Apakah penjelasan analogi "Detektif" untuk Bayesian ini membantu kamu memvisualisasikan cara kerjanya dibanding metode lain?

Powered by Gemini All Chat Downloader